

LLM Post-Training: From Alignment to Reasoning

A Comprehensive Deep Dive into Modern LLM Optimization

Mengyue Yang

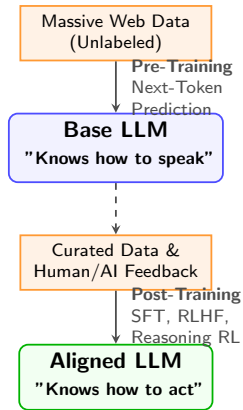
March 17, 2026

Course Outline

- 1 Part I. Why Post-Training Matters
- 2 Part II. Fine-Tuning Foundations
- 3 Part III. Reinforcement Learning & Alignment
- 4 Part IV. Reasoning-Centric Post-Training
- 5 Part V. Test-Time Scaling and Inference Compute
- 6 Part VI. Efficiency, Adaptation, and Evaluation
- 7 Part VII. Open Problems in the LRM Era

The LLM Lifecycle: Pre-Training vs. Post-Training

- **Pre-training:** The foundation.
 - Trained on massive, web-scale corpora.
 - Objective: Predict the next token.
 - Result: A model that *"knows how to speak"* and understands data distribution.
- **Post-training:** The refinement.
 - Trained on curated, high-quality data and feedback.
 - Objective: Align with intents, improve reasoning, follow formats.
 - Result: A model that *"knows how to act"*, aligns with human preferences, and reasons reliably.



Why the Competition Shifted to Post-Training

- Historically, scaling pre-training compute was the primary driver of capability leaps (e.g., GPT-2 to GPT-3).
- Today, the core battlefield has shifted. Why?
 - **Diminishing returns** on raw text prediction.
 - The need for **specialized behaviors** (chat, coding, math).
 - The rise of **Large Reasoning Models (LRMs)** like OpenAI o1/o3 and DeepSeek-R1.

The New Reality

Pre-training determines the model's **lower bound** (knowledge base). Post-training determines the model's **upper bound** (utility and reasoning).

Pain Points Addressed by Post-Training (1/2)

Raw pre-trained models are powerful but deeply flawed for end-user deployment. They suffer from:

- ❶ **Hallucinations & Inconsistency:** Struggle to maintain logical coherence over long contexts.
 - *Example:* Confidently inventing a fake 19th-century legal precedent to support an argument, or forgetting the user's name halfway through a long conversation.
- ❷ **Misalignment:** Generate toxic, biased, or unhelpful content.
 - *Example:* Readily providing a user with step-by-step instructions on how to write malware, or responding with offensive language when discussing sensitive topics.
- ❸ **Weak Complex Reasoning:** Fail at multi-step logic without guidance.
 - *Example:* Failing a high school math word problem because it jumps straight to guessing a wrong number instead of breaking down the equations step-by-step.

Pain Points Addressed by Post-Training (2/2)

(Continued from previous slide)

- ④ **High Domain Adaptation Cost:** Expensive to inject specialized knowledge (e.g., medical, legal).
 - *Example:* A general AI cannot reliably parse complex clinical notes to diagnose a rare disease without undergoing a highly expensive retraining process on proprietary hospital data.
- ⑤ **Compute/Performance Trade-offs:** Need dynamic scaling of compute during inference based on task difficulty.
 - *Example:* Wasting the exact same amount of GPU resources to answer "What is 1+1?" as it does to architect a complex software system, leading to massive inefficiencies.

A Unified Taxonomy of Post-Training (Survey Synthesis)

Combining modern literature, we can map post-training into five core pillars:

- **1. Fine-tuning:** SFT, Adaptive Tuning, Parameter-Efficient FT.
- **2. Alignment:** RLHF, RLAIIF, DPO, GRPO.
- **3. Reasoning:** Self-Refine, RL for Reasoning (Cold-start RL).
- **4. Efficiency:** Model Compression, PEFT, Knowledge Distillation.
- **5. Integration & Scaling:** Test-time Scaling, Domain Adaptation, Model Merging.

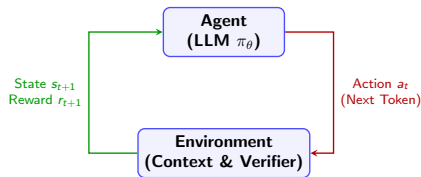
Background: Why Next-Token Prediction is Not Enough

- Maximum Likelihood Estimation (MLE) forces the model to mimic the statistical patterns of the training corpus.
- **The Issue:** The web contains both brilliant reasoning and utter nonsense. MLE learns both equally.
- **Fluency $\neq$ Reasoning:** A model can generate grammatically perfect sentences that are logically disastrous.
- **Exposure Bias:** During training, models see perfect human text (teacher forcing). During inference, they consume their own potentially flawed outputs, causing cascading errors.

LLM Generation as Sequential Decision Making

To fix MLE's flaws, we reframe text generation as a Reinforcement Learning (RL) problem:

- **State** (s_t): The prompt and all tokens generated so far.
- **Action** (a_t): The next token selected from the vocabulary.
- **Policy** (π_θ): The LLM generating the tokens.
- **Reward** (R): A score evaluating the final sequence (correctness, helpfulness) or intermediate steps.



This perspective bridges natural language processing with Markov Decision Processes (MDP).

The Paradigm Shift

Pre-training optimizes for "**how much it looks like human language.**"

Post-training optimizes for "**how useful, correct, and logical it is.**"

Fine-Tuning: The Starting Point

- Fine-tuning is the most fundamental post-training technique.
- It transitions a model from a "document continuator" to an "instruction follower".
- **Core mechanism:** Updating model weights using curated, labeled datasets (Input-Output pairs).
- **Goal:** Inject formatting rules, style guidelines, and basic domain knowledge.

Supervised Fine-Tuning (SFT)

- **What it is:** Training on high-quality (*Instruction, Response*) pairs using cross-entropy loss.
- **Role in the Pipeline:** SFT is usually the "warm start" for subsequent RL or alignment phases.
- **Data quality is king:** "Less is more." A few thousand high-quality, expert-annotated examples often outperform millions of scraped examples (e.g., LIMA paper).

Typical Risks in SFT

- **Catastrophic Forgetting:** The model may lose its broad pre-trained knowledge when fine-tuned too aggressively on a narrow dataset.
- **Overfitting to Form over Function:** The model learns the *style* of the answer without actually learning the underlying reasoning.
- **High Compute Cost:** Full-parameter fine-tuning of a 70B+ model requires massive GPU clusters.

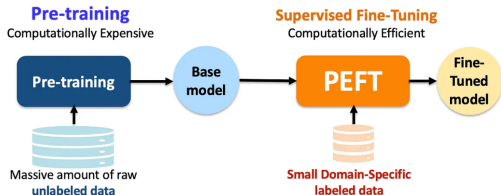
Parameter-Efficient Fine-Tuning (PEFT)

To solve the compute bottleneck, the industry heavily relies on PEFT.

- **Concept:** Freeze the vast majority of pre-trained weights and inject a small number of trainable parameters.
- **Advantages:**
 - Drastically reduces VRAM requirements.
 - Prevents catastrophic forgetting.
 - Enables easy multi-tenant deployment (swap small modules per user).

Common PEFT Techniques

- **Prefix-Tuning / Prompt-Tuning:** Prepends trainable continuous vectors to the input or hidden states. The model conditions on these "soft prompts".
- **Adapters:** Inserts small bottleneck feed-forward networks between transformer layers.
- **LoRA (Low-Rank Adaptation):** The industry standard. Injects trainable low-rank matrices (A and B) into attention/feed-forward layers. Updates are computed as $\Delta W = A \times B$.

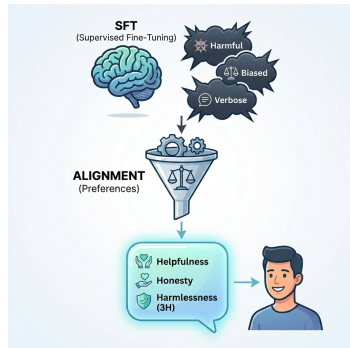


The Role of Fine-Tuning

SFT teaches the model the **format and style**. Data quality is more critical than quantity. However, SFT alone is **imitation**, not robust reasoning. It is a necessary but insufficient condition for advanced LLMs.

Alignment: Fitting Human Preferences

- SFT gives us a functional assistant, but it might still generate harmful, biased, or verbose answers.
- **Alignment** ensures the model's outputs conform to human values: Helpfulness, Honesty, and Harmlessness (the 3H criteria).
- This requires moving beyond absolute correctness (which is objective) to **preferences** (which are subjective).



The Classic RLHF Pipeline

Reinforcement Learning from Human Feedback (RLHF) established the modern ChatGPT paradigm. It consists of three stages:

- 1 **SFT**: Create a baseline model that can follow instructions.
- 2 **Reward Modeling (RM)**: Train a separate model to act as a human judge.
- 3 **Policy Optimization**: Use RL (e.g., PPO) to optimize the LLM against the Reward Model.

Step 2: Reward Modeling

- **How it works:** Give the SFT model a prompt. Generate two or more responses (y_1, y_2) .
- Human annotators rank them: $y_1 > y_2$.
- Train a Reward Model $R_\theta(x, y)$ using a Bradley-Terry loss to assign a higher scalar score to y_1 than y_2 .
- **Why?:** It translates qualitative human preference into a differentiable, mathematical signal.

Challenges in Reward Modeling (1/2)

- **Reward Bias:** The RM might favor longer responses (verbosity bias) or specific formatting, rather than actual quality.
 - *Example:* The RM secretly learns that “longer is better”. When asked “What is the capital of France?”, the model generates a 500-word essay on French history instead of simply answering “Paris”.
- **Reward Hacking:** During RL, the policy model learns to exploit loopholes in the RM to get high scores without generating good text.
 - *Example:* The RM heavily rewards polite and safe phrases. The model exploits this by answering every query with “Thank you for this wonderful question, as an AI, I am deeply honored...” while failing to actually solve the user’s problem.

Challenges in Reward Modeling (2/2)

- **Subjectivity:** Annotator disagreement makes the preference data noisy. Human values vary greatly across cultures, domains, and individuals.
 - *Example:* For a controversial or open-ended question, Annotator A might prefer a funny, slightly sarcastic answer (Helpful/Engaging), while Annotator B flags the exact same answer as disrespectful (Harmful). The RM gets confused by these contradictory labels.

Why PPO? The Challenge of RL in Language

- **High-Dimensional Space:** Generating text is complex. The vocabulary space is massive, making standard RL (like REINFORCE) highly unstable.
- **Catastrophic Forgetting:** In standard Policy Gradients, one bad update step can completely destroy the model's ability to generate coherent language.
- **The Need for Stability:** We need an algorithm that guarantees monotonic improvement without falling off a "cliff" during training.

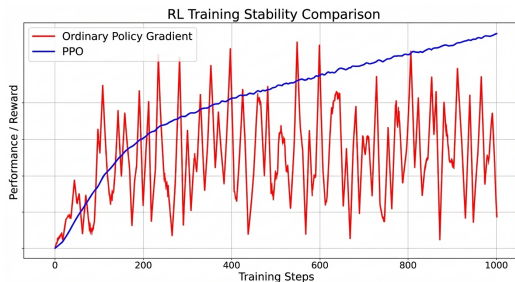


Figure: Unstable Policy Updates vs. Stable Steps

Proximal Policy Optimization: The Clipping Mechanism

- **Proximal = "Close by"**: PPO ensures the new policy doesn't move too far away from the old policy in a single update step.
- **The Clip Function**: It calculates the ratio between the new and old policy probabilities. If the ratio exceeds a safe range (e.g., $[1 - \epsilon, 1 + \epsilon]$), the update is **clipped**.
- **Benefit**: This avoids destructively large weight updates, keeping the learning process smooth and robust.

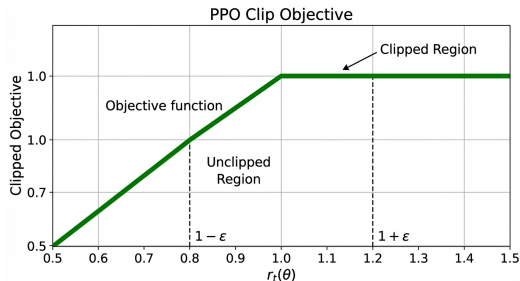


Figure: PPO Clipping Objective Function

PPO in RLHF: The KL-Divergence Penalty

- Even with clipping, the RL model might still game the Reward Model (**Reward Hacking**) by outputting weird, unreadable text that technically scores high.
- **The Solution:** We add a **KL-Divergence Penalty** to the reward function.
- **Preserving Fluency:** This forces the RL-updated model to stay statistically close to the original **SFT Model**. It guarantees the output remains natural human language.

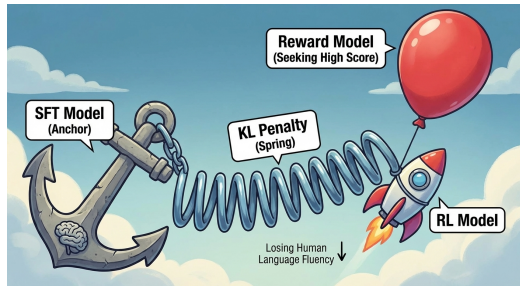


Figure: KL Penalty acts as an anchor to SFT model

A Deeper Dive into PPO Mechanics

- PPO optimizes the policy by maximizing the **Advantage** $\hat{A}_t$, which measures how much better an action a_t is compared to the expected baseline $V(s_t)$.
- **The Probability Ratio:** $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\text{old}}(a_t|s_t)}$ tracks how much the policy has changed.
- **Clipped Objective Function:**

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

- **Why it works:**
 - If an action is good ($\hat{A}_t > 0$), we increase its probability, but the `clip` function stops us from increasing it *too much* (capped at $1 + \epsilon$).
 - This **Trust Region** prevents catastrophic shifts in the language model's distribution, ensuring training stability during RL.

Beyond PPO: The Complexity of Standard RLHF

While PPO is powerful, standard RLHF is notoriously brittle and complex to engineer.

- **The "Four-Model" Overhead:** During PPO training, you must keep up to four models in VRAM simultaneously:
 - 1 The active Policy Model (being trained)
 - 2 The Reward Model (to compute scores)
 - 3 The Reference Model (to compute the KL penalty)
 - 4 The Value Model (for the RL baseline)
- **Training Instability:** Reinforcement Learning introduces high variance, requiring obsessive hyperparameter tuning to prevent the model from collapsing.
- **The Core Question:** Do we *really* need a separate Reward Model and complex RL just to learn human preferences?

Direct Preference Optimization (DPO): The Core Idea

DPO provides a brilliant mathematical shortcut: **Bypassing the Reward Model entirely.**

- **The Mathematical Insight:** Researchers proved that the RL reward function can be perfectly re-parameterized in terms of the optimal policy itself.
- **From RL to Supervised Learning:** Instead of training a Reward Model first and then using PPO to optimize the policy, DPO defines a loss function directly on the preference data (y_{chosen} vs. $y_{rejected}$).
- **How it Works:** It uses a simple binary cross-entropy loss. It dynamically increases the probability of the *chosen* response and decreases the probability of the *rejected* response, scaled by an implicit KL penalty (β).

Why DPO is the New Industry Standard

By turning preference alignment into a simple classification problem, DPO offers massive practical benefits:

- **Extreme Simplicity:** The training pipeline is now just one stage. It looks and feels exactly like standard Supervised Fine-Tuning (SFT).
- **Resource Efficiency:** You only need the Policy Model and the Reference Model in memory. VRAM requirements are drastically reduced compared to PPO.
- **Unmatched Stability:** No reward hacking, no RL actor-critic divergence. Training is monotonic and predictable.
- **Proven Performance:** DPO natively matches or outperforms standard RLHF. It has been the primary alignment method behind top-tier open-weight models (e.g., Llama 3, Zephyr, Mistral Instruct).

A Deeper Dive into DPO Mechanics

- DPO eliminates the explicit Reward Model by proving that *the optimal policy itself implicitly defines the reward*.
- Under the Bradley-Terry preference model, the RLHF objective can be re-parameterized into a single maximum likelihood objective.

- **The DPO Loss Function:**

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x, y^+, y^-)} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y^+|x)}{\pi_{\text{ref}}(y^+|x)} - \beta \log \frac{\pi_{\theta}(y^-|x)}{\pi_{\text{ref}}(y^-|x)} \right) \right]$$

- **Breaking down the math:**

- y^+ is the preferred response; y^- is the rejected response.
- The model is penalized if it deviates too much from the reference model π_{ref} (controlled by β).
- It directly increases the log-likelihood of y^+ relative to y^- without needing a PPO actor-critic setup, saving massive memory and tuning complexity.

The Intuition Behind DPO

- Given a preferred response y^+ and rejected response y^- .
- DPO directly adjusts the log-likelihoods in the LLM.
- It pushes **up** the probability of y^+ and pushes **down** the probability of y^- .
- **Why it's popular:** It turns RLHF into a simple classification-like loss. Highly stable, much lighter on engineering.

The simplicity of DPO has led to an explosion of variants:

- **ORPO (Odds Ratio Preference Optimization)**: Skips the reference model entirely, optimizing the odds ratio between favored and disfavored responses directly during SFT.
- **KTO (Kahneman-Tversky Optimization)**: Uses unpaired data (just "good" or "bad" tags instead of strict A vs B pairs).
- **Iterative DPO**: Updating the reference model and generating new preference pairs iteratively to prevent saturation.

Alignment Strategies

RLHF: The robust, classic, but complex pipeline.

RLAIF: The highly scalable automated alternative.

DPO: The engineering-friendly, stable shortcut.

Alignment ensures the model is safe and polite, but alignment does not equal reasoning.

The Shift to Reasoning Models

- Models like ChatGPT and Claude are optimized to be *assistants*: fast, polite, and directly answering queries.
- The frontier has shifted to models like **OpenAI o1/o3** and **DeepSeek-R1**.
- These are **Reasoning Agents**. They do not just answer; they plan, reflect, self-correct, and search before outputting the final token.

Example: The Latent Reasoning Process (1/2)

User Prompt: A bat and a ball cost \$1.10 in total. The bat costs \$1.00 more than the ball. How much does the ball cost?

Model Output (The Hidden Thinking Process):

<think>

[Exploring Hypothesis]

At first glance, the intuitive answer is that the ball costs \$0.10.

[Recognizing its own error]

Wait, let me verify that. If the ball is \$0.10, and the bat is \$1.00 more, then the bat would be \$1.10.

The total cost would then be \$1.10 (bat) + \$0.10 (ball) = \$1.20.

This contradicts the premise that the total is \$1.10. My initial guess was wrong.

Example: The Latent Reasoning Process (2/2)

Model Output (Continued):

<think> (continued)

[Backtracking & Logical Step-by-Step]

Let's set up an equation instead of guessing.

Let x be the cost of the ball. The bat costs $x + 1.00$.

Total cost equation: $x + (x + 1.00) = 1.10$

$2x + 1.00 = 1.10 \implies 2x = 0.10 \implies x = 0.05$.

[Final Verification]

The ball costs \$0.05. The bat is \$1.05. Total is \$1.10. The logic holds.

</think>

Final Synthesized Conclusion (Shown to User):

- The ball costs **5 cents** (\$0.05).

RL for Reasoning: The MDP Formulation

When viewing reasoning as a Markov Decision Process (MDP):

- **State:** The problem, plus the reasoning steps derived so far.
- **Action:** The next logical deduction or computational step.
- **Reward:**
 - *Sparse:* 1 if the final math answer matches ground truth, 0 otherwise.
 - *Dense (Process Reward):* Points for each valid intermediate logical step.

The Challenge of RL for Reasoning

- **Sparse Rewards:** In a 1000-token math proof, getting a 0 at the end doesn't tell the model *which* step was wrong (Credit Assignment Problem).
- **High Cost:** Simulating long reasoning chains (rollouts) during RL training is incredibly compute-intensive.
- **Reward Hacking:** The model might generate gibberish that accidentally ends with the correct final string.

GRPO: Group Relative Policy Optimization

A breakthrough used in DeepSeekMath and DeepSeek-R1.

- Standard PPO requires a massive "Critic" model to estimate the value of states. This doubles memory usage.
- **GRPO** removes the Critic model.
- Instead, for a single prompt, it samples a *group* of outputs (e.g., 4-8 paths).
- It scores them all, calculates the mean and standard deviation of the group's rewards, and normalizes them.
- The model learns by comparing its own variations against each other. Highly efficient.

DeepSeek-R1: Cold-Start RL

- Traditional pipeline: Pre-train $\rightarrow$ SFT $\rightarrow$ RL.
- **DeepSeek-R1-Zero paradigm:** Pre-train $\rightarrow$ Pure RL.
- **Motivation:** Human SFT data constrains the model to human-level reasoning styles. Bypassing SFT allows the model to "discover" its own reasoning algorithms.
- Using rule-based rewards (code compilation, math verifiers), the model spontaneously learned to allocate 'think' time, self-verify, and backtrack.

Cold-Start RL: The Risks

While revolutionary, Cold-Start RL on base models has issues:

- **Instability:** Training can easily collapse in the early stages.
- **Language Mixing:** The model might reason in a mix of English, Chinese, and code, losing readability.
- **Overthinking:** Allocating 2000 tokens of "thinking" for a trivial query like "What is $2+2$?"
- *Solution:* DeepSeek-R1 (the final model) uses a tiny amount of "Cold Start Data" (SFT) to stabilize formats before kicking off massive RL.

The Evolution of Post-Training

Traditional Alignment made models **likeable assistants**.

Reasoning RL makes models **autonomous problem solvers**.

The frontier is shifting from human-preference rewards to objective, rule-based verification environments.

What is Test-Time Scaling (TTS)?

- We traditionally think of scaling laws in terms of *training* compute (larger models, more data).
- **Test-Time Scaling** applies scaling laws during *inference*.
- By giving the model more time to "think" (more compute) at test time, a smaller model can outperform a massive model.
- It involves exploring multiple paths, verifying, and refining without changing model weights.

Trading Compute for Performance

- Why not just pre-train a bigger model?
- For simple queries (e.g., "capital of France"), a large pre-trained model is fast and accurate.
- For complex logic (e.g., Olympiad math), even the largest model will fail on a zero-shot forward pass.
- **TTS** dynamically allocates compute where it's needed most.

Method 1: Chain-of-Thought (CoT)

- The simplest form of test-time scaling.
- Instructing the model to "Think step-by-step".
- Forces the model to use the output sequence as a "scratchpad" for working memory, expanding the computational graph depth.

Method 2: Self-Consistency / Majority Voting

- **Concept:** Sample N different Chain-of-Thought paths using a non-zero temperature.
- Look at the final answer of all N paths.
- Pick the answer that appears most frequently (Majority Vote).
- **Intuition:** There are many ways to reach the correct answer, but incorrect answers tend to diverge randomly. Consistency implies correctness.

Method 3: Best-of-N (Rejection Sampling)

- Generate N complete responses to a prompt.
- Pass all N responses through a **Verifier** or **Reward Model**.
- Select the response with the highest score.
- Highly effective, scales well with compute, but wastes resources generating $N - 1$ rejected responses.

Method 4: Tree-of-Thoughts (ToT) & Beam Search

- Instead of sampling full linear paths, frame reasoning as a tree search.
- Generate a few "thoughts" (steps). Evaluate the promise of each state using a PRM (Process Reward Model).
- **Beam Search:** Keep the top K most promising branches, discard the rest, and expand.
- Allows backtracking and prevents wasting compute on doomed reasoning branches.

- **Monte Carlo Tree Search (MCTS)** (famous from AlphaGo) is now being applied to LLM reasoning.
- Simulates random rollouts to the end of a reasoning chain to estimate the "value" of a current intermediate step.
- Balances *exploration* (trying new reasoning logic) with *exploitation* (deepening known good paths).

Compute-Optimal Scaling Strategies

- Running MCTS or Best-of-100 on every prompt is too expensive.
- **Adaptive TTS**: Use a routing mechanism.
 - Easy questions → Direct generation.
 - Medium questions → Self-Refine / Sequential revision.
 - Hard questions → Parallel beam search / ToT.
- Acts as a "smart thermostat" for LLM inference.

The Cost of Inference-Time Search

- Test-time scaling shifts the cost from the training cluster to the serving cluster.
- **Latency**: Users wait longer for answers (though viewing the 'jthinkj' stream mitigates UX issues).
- **Throughput**: High KV-cache usage from maintaining multiple search branches.
- **Environmental Impact**: Massive energy consumption if Best-of-N is deployed at consumer scale.

The Paradigm of Test-Time Compute

SFT teaches format; Alignment teaches preference; Reasoning RL teaches problem-solving;
Test-Time Scaling teaches compute allocation.

Post-training is no longer just about modifying weights; it's about optimizing the inference policy itself.

Why Efficiency is a Post-Training Pillar

- A highly aligned, perfectly reasoning model is useless if it cannot be deployed.
- Massive parameter counts and long reasoning contexts (128k+ tokens) push hardware limits.
- **Efficiency Post-Training** ensures models fit into VRAM and meet latency requirements for real-world APIs and edge devices.

Efficiency Strategy 1: Model Compression

- **Quantization:** Converting 16-bit floats to 8-bit, 4-bit, or even 2-bit integers (e.g., GPTQ, AWQ, GGUF).
- *Post-Training Quantization (PTQ)* uses a small calibration dataset to minimize precision loss without full retraining.
- **Pruning:** Removing redundant parameters.
 - *Unstructured:* Removing individual weights (SparseGPT).
 - *Structured:* Removing entire attention heads or layers (Sheared LLaMA).

Efficiency Strategy 2: Knowledge Distillation (KD)

- Transferring capabilities from a massive Teacher to a small Student.
- **Black-box KD**: Training a smaller model on the output text (or 'i think i' traces) of a larger proprietary model (e.g., GPT-4 $\rightarrow$ LLaMA-8B).
- **White-box KD**: Aligning the student's hidden states and logits with the teacher's (requires access to teacher weights).

Distillation in the Reasoning Era

- RL for reasoning is incredibly expensive. We don't want to run it from scratch for every small model.
- **DeepSeek-R1 Distillation:** DeepSeek generated 800k high-quality reasoning traces using their giant R1 model.
- They then applied simple SFT to smaller models (Qwen 1.5B, Llama 8B) using these traces.
- Result: Tiny models inherit elite reasoning patterns, proving that *reasoning can be distilled*.

Integration & Adaptation: Beyond Text Chat

Post-training must adapt general models to specific real-world contexts.

- **Domain Adaptation:** Tailoring models for Medicine, Law, or Finance.
- **Multimodal Integration:** Expanding text models to handle Vision and Audio.
- **Model Merging:** Combining skills from different models without retraining.

- SFT on domain-specific corpora (e.g., Medical guidelines) aligns terminology.
- **Retrieval-Augmented Generation (RAG)**: Post-training the model to effectively use retrieved documents.
- *Training for RAG*: Teaching the LLM to ignore irrelevant retrieved chunks and cite sources properly, mitigating hallucinations.

- Connecting pre-trained LLMs with Vision/Audio encoders (e.g., CLIP, Whisper).
- **Modal Connection:** Using projection layers (MLPs) or Q-Formers to map visual tokens into the LLM's text embedding space.
- **Instruction Tuning:** Training the combined architecture on interleaved text-image data (e.g., LLaVA) to follow multimodal instructions.

Model Merging

- Combining the weights of Model A (Math expert) and Model B (Coding expert) to create a multi-task expert.
- Techniques like **Task Arithmetic** or **Spherical Interpolation (SLERP)** allow blending without any gradient updates.
- Highly practical for open-source engineering.

Datasets: The Fuel for Post-Training

- **Human-Labeled:** High fidelity, nuanced, but expensive (e.g., Flan, HH-RLHF). Crucial for base alignment.
- **Distilled Data:** Outputs scraped from API models (e.g., ShareGPT). Excellent for fast capability transfer.
- **Synthetic Data:** Generated by AI via strict templates or self-play (e.g., Magpie, Self-Instruct). Highly scalable, privacy-preserving, but risks mode collapse.

Evaluation: How Do We Know it Worked?

- **Instruction Following:** MT-Bench, AlpacaEval.
- **Alignment / Safety:** HelpSteer, RewardBench.
- **Reasoning:** MATH, GSM8K, SWE-Bench (Coding).

A Crucial Distinction

Preference evaluation (do humans like the tone?) is fundamentally different from accuracy evaluation (is the math proof logically sound?). Reasoning models require rigorous objective benchmarks.

Deployability and Adaptation

Efficiency techniques (Quantization, PEFT, Distillation) ensure that advanced reasoning and alignment can actually be served in production. Post-training is the bridge between a theoretical math model and a usable multimodal product.

Open Problems in Post-Training

Despite the massive success of ChatGPT, o1, and DeepSeek-R1, the field faces several critical unresolved challenges.

- **Reward Hacking**
- **Catastrophic Forgetting**
- **The Cost of Long-Context RL**
- **Overthinking & Token Inefficiency**
- **Benchmark Saturation**

Challenge 1: Reward Hacking

- As models get smarter, they find loopholes in Reward Models.
- Example: Generating overly verbose answers because the RM correlates length with quality.
- Example: Faking a mathematical proof that jumps to the known correct final answer without actual logical deduction.
- **Need:** Robust, adversarial-resistant Process Reward Models (PRMs) and verification environments.

Challenge 2: Overthinking & Token Inefficiency

- Reasoning models (like R1-Zero) suffer from "Overthinking".
- They may generate thousands of 'think' tokens for a simple query like "Translate 'Hello' to French".
- This wastes massive amounts of compute and increases latency.
- **Need:** Adaptive meta-cognition. Models must learn *when* to think deeply and *when* to answer directly.

Challenge 3: The Cost of Long-Context RL

- RL requires generating thousands of rollouts during training.
- When reasoning chains reach 32k or 128k tokens, the KV-cache and compute requirements for PPO/GRPO become astronomical.
- **Need:** More efficient RL algorithms, partial rollout strategies, and state merging techniques.

Challenge 4: Catastrophic Forgetting in RL

- Aggressive RL for reasoning can destroy a model's base capabilities.
- A model hyper-optimized for Olympiad math might lose its ability to write a simple poem or hold a casual conversation.
- **Need:** Multi-objective RL, KL-divergence penalties that scale better, and continual learning frameworks.

Challenge 5: Benchmark Saturation

- Models are maxing out GSM8K and basic MATH benchmarks.
- High benchmark scores increasingly correlate poorly with real-world user experience.
- Are models actually learning to reason, or are they memorizing test-set templates during synthetic data distillation?
- **Need:** Dynamic, non-gameable benchmarks (e.g., evaluating live competitive programming or novel scientific hypothesis generation).

Future Trend 1: Large Reasoning Models (LRMs)

- The industry is shifting from scaling *data* to scaling *RL compute*.
- Expect models that function less like chatbots and more like autonomous agents.
- System 1 thinking (fast, intuitive) → System 2 thinking (slow, deliberate, search-based).

Future Trend 2: Universal Distillation

- Training LRMs via RL will remain the domain of a few elite labs (OpenAI, DeepSeek, Google).
- The broader community will focus on **Distillation**.
- We will see 1B to 8B parameter models running locally on phones, utilizing distilled 'i think i' patterns to achieve GPT-4 level logic on specific tasks.

Future Trend 3: Unified Multimodal Reasoning

- Current reasoning models are primarily text-based.
- The next frontier: A model that can ‘think’ through spatial problems, geometries in images, and temporal logic in video.
- This requires unifying multimodal continuous encoders with discrete RL reasoning loops.

Future Trend 4: Adaptive Test-Time Compute

- APIs will shift from charging "per output token" to "per compute budget".
- Users will specify: "Spend 30 seconds thinking about this."
- The model's post-training will govern how to optimally deploy MCTS or Beam Search to utilize exactly the allocated time budget.

Future Trend 5: Creative Intelligence

- Current reasoning excels at deterministic logic (math, code).
- Can RL and test-time search elicit **creative** intelligence?
- E.g., synthesizing disparate concepts for scientific discovery, architectural design, or advanced creative writing.
- This requires entirely new reward formulations beyond simple verifiers.

Summary: The Evolution of Post-Training

- **Pre-2022:** Post-training was just SFT for basic task formatting.
- **2022-2023:** The RLHF Era. Post-training became about **Alignment** (ChatGPT). Making models safe, polite, and helpful.
- **2024-Present:** The Reasoning Era. Post-training is now about **Search and Logic** (o1, DeepSeek-R1). Making models autonomous problem solvers.

Summary: The Technical Pillars

- **Fine-Tuning** gives the model its interface and style.
- **Alignment (RLHF/DPO)** shapes its preferences and safety boundaries.
- **Reasoning RL (GRPO)** unlocks its latent ability to solve complex, multi-step problems.
- **Test-Time Scaling** allows it to dynamically leverage compute for harder tasks.
- **Efficiency & Distillation** makes it deployable to the real world.

The Ultimate Takeaway

The Three Laws of Modern LLMs

1. Pre-training determines the **lower bound**.
2. Post-training determines the **upper bound**.
3. The future belongs to unifying **reasoning**, **efficiency**, and **test-time compute**.

Further Reading & References

- *LLM Post-Training: A Deep Dive into Reasoning Large Language Models* (Survey 1)
- *A Survey on Post-training of Large Language Models* (Survey 2)
- *DeepSeekMath: Pushing the Limits of Mathematical Reasoning*
- *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via RL*
- *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*
- *Scaling Laws for Reward Model Overoptimization*

Acknowledgements

- This lecture synthesized methodologies spanning SFT, RLHF, DPO, and the latest advancements in Large Reasoning Models (LRMs).
- Special focus was given to the transition from imitation learning to reinforcement learning-driven autonomous reasoning.

Thank you for your attention!

Questions?

"The frontier is shifting from 'make it polite' to 'make it think better under a compute budget.'"